M. Yusuf Aykut
Istanbul University, Department of Computer Science

Reward-Free World-Model Pretraining for Sample-Efficient Reinforcement Learning

TL;DR

In earlier work, I cast a design problem as RL: a DQL agent interacted with a fast simulator to optimize a constrained objective, and a simple multi-head variant stabilized training. I am now pursuing a general, domain-agnostic recipe: (i) learn a reward-free world model (encoder, latent dynamics, decoder) purely from (s, a, s') logs; (ii) optimize the policy inside that model via short imagined rollouts using a surrogate objective that encodes task goals and constraints; and (iii) periodically refresh the model with brief real interactions. This workflow reduces expensive real environment calls, stabilizes learning under constraints, and yields a reusable model that adapts quickly to new preferences or objectives.

Before Work and Backgorund

Because the task is inherently multi-objective and the **environment** has a **high-dimensional state space**, steering exploration toward the target—maximize J_{ph} to $AVT \geq 25$ —is non-trivial. The resulting reward landscape is **non-convex** and partly **sparse**: small moves that help one objective can harm the other, and occasional constraint violations are inevitable during exploration. This shows up clearly in the reward trace: even as the running mean increases, some episodes terminate with negative cumulative return, producing the characteristic zig-zag (saw-tooth) pattern. These downward spikes are not training pathologies; they are the observable footprint of the **trade-off structure** and the **exploration–exploitation** tension induced by optimizing conflicting objectives under a hard AVT constraint.

Figure 1

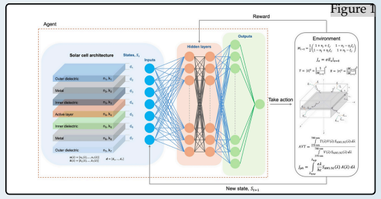


Figure 2

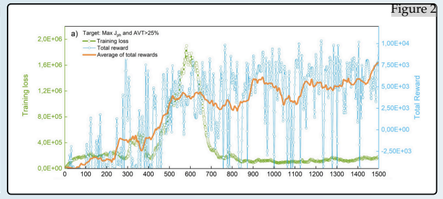
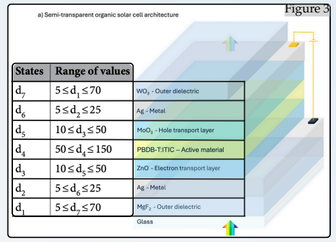


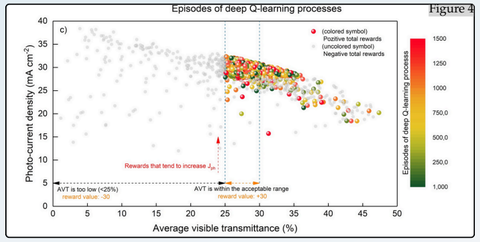
Figure 4 plots AVT vs. J_{ph} for the ST-OSC designs explored by the DQN. Because the objectives conflict, the upper-right envelope of points approximates a **Pareto front** (non-dominated designs). With a reward that maximizes J_{ph} while encouraging $AVT \geq 25$ (and higher), the agent settles on a compromise along this front; changing the reward weights would move the selected point along the same **trade-off**.

Figure 3



States	Range of values	
d_7	$5 \leq d_1 \leq 70$	WO_3 - Outer dielectric
d_6	$5 \leq d_2 \leq 25$	Ag - Metal
d_5	$10 \leq d_3 \leq 50$	MoO_3 - Hole transport layer
d_4	$50 \leq d_4 \leq 150$	PBDB-T/ITIC – Active material
d_3	$10 \leq d_5 \leq 50$	ZnO - Electron transport layer
d_2	$5 \leq d_6 \leq 25$	Ag - Metal
d_1	$5 \leq d_7 \leq 70$	MgF_2 - Outer dielectric

Figure 4



Cetinkaya, C., Cokduygulular, E., **Aykut, M.Y.** et al. Artificial intelligence-empowered functional design of semi-transparent optoelectronic and photonic devices via deep Q-learning. Sci Rep 15, 13508 (2025). https://doi.org/10.1038/s41598-025-94586-x

Problem Definition

Many real-world control and design problems are both expensive to interact with and intrinsically multi-objective: agents must push a primary objective while keeping one or more constraints satisfied. In standard model-free RL, the agent repeatedly queries the environment, which leads to high sample cost and unstable learning curves when rewards are sparse, discontinuous, or tied to constraint satisfaction.

Figure 5

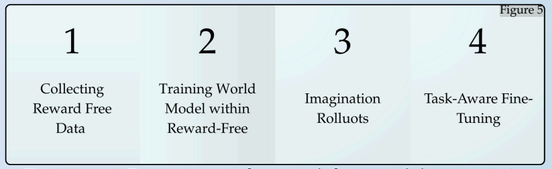


Figure 5. *Four steps of reward-free model pretraining to task-aware adaptation*

What we will do?

- Learn a world model without rewards, using only (s,a,s') logs.
- Decode task features from that model (not rewards), then optimize the policy inside the model with short imagination rollouts; we add brief real runs only to refresh.
- Goal: sample-efficient, constraint-aware multi-objective control with far fewer real environment calls.

How the mechanisms serve this?

- Dreamer-style mechanism (Fig. 6): encoder + latent dynamics align prior/posterior (KL); decoder predicts task features. This figure shows the mechanism, not the whole loop.
- **Surrogate objective from decoded features:** $f = r(\hat{\psi}) - \sum_k \lambda_k c_k(\hat{\psi})$ lets us train for goals/constraints **without external rewards.**
- Imagination + gating: plan in the model; mask unsafe/uncertain rollouts → fewer wasteful or risky real trials.
- Periodic real refresh: small bursts of real data prevent model drift and keep learning grounded.

Figure 6

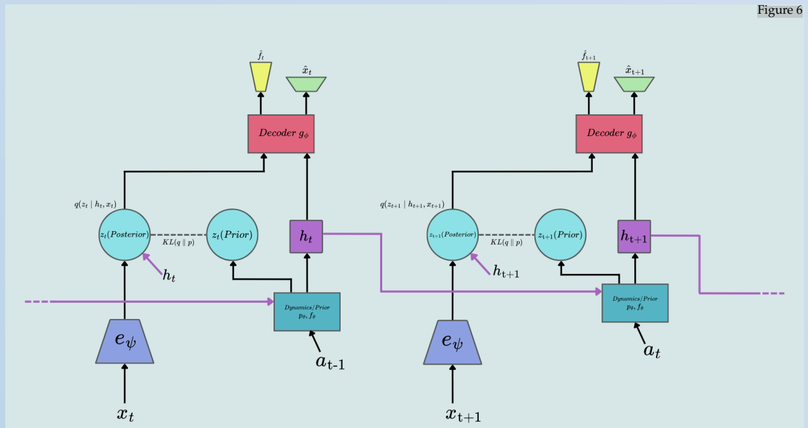


Figure 6. *At time t, x_t is encoded to a posterior latent z_t ; dynamics with (h_{t-1} , a_{t-1}) produce a prior z_t and next hidden h_t aligned by $\mathrm{KL}(q\|p)$. The decoder uses (h_t, z_t^{post}) to predict (optionally) for self-supervised losses; h_t carries forward. This stage is reward-free; task-aware policy learning for Recurrent State-Space Model later uses imagined rollouts inside the model.*

Reward-Free World-Model RL

Input: env E; dataset $D \leftarrow \varnothing$; model $(e_\psi, p_\theta, f_\theta, g_\phi)$; policy π_η; value V_ω

for *iter* $= 1$ **to** T **do**
  if *iter* $= 1$ ***or*** *iter* $\bmod M = 0$ **then**
    collect reward-free episodes with π_{exp} in E; append (s, a, s') to D
  end if
  train world model on D by minimizing:
    $\mathrm{KL}(q_\psi(z_t|s_t, h_{t-1}) \,\|\, p_\theta(z_t|h_{t-1}, a_{t-1})) + \mathcal{L}_{\text{feat}}(g_\phi(h_t, z_t),\ f(s_t))$;
  for $n = 1$ **to** $N_{imagine}$ **do**
    sample (s, h) from D; $z \leftarrow e_\psi(s)$;
    for $k = 1$ **to** K **do**
      $a_k \sim \pi_\eta(\cdot|h_k)$;
      $z_{k+1} \sim p_\theta(z_k, a_k)$;
      $h_{k+1} \leftarrow f_\theta(h_k, z_{k+1}, a_k)$;
      $\hat{f}_k \leftarrow g_\phi(h_k, z_k)$;
      $\hat{r}_k \leftarrow R(\hat{f}_k) - \sum_i \lambda_i [c_i(\hat{f}_k)]_+$;
      if *high uncertainty / predicted constraint violation* **then**
        break
      end if
    end for
    update V_ω and π_η from imagined returns
  end for
end for

Pseudo-code. *This algorithm first gathers environment interactions without relying on external rewards, then trains a generative world model to capture the system's dynamics. By simulating imagined trajectories within this learned model, it enables the agent to optimize its policy and value function efficiently—even when real rewards are unavailable or costly. The result is a framework that decouples representation learning from task-specific optimization, unlocking sample-efficient RL in challenging domains.*

[1] D. Hafner, T. Lillicrap, M. Norouzi, J. Ba. Learning Latent Dynamics for Planning from Pixels (PlaNet). arXiv:1811.04551, 2019. (Introduces the Recurrent State-Space Model)
[2] C. Chen, J. Yoon, Y. Wu, S. Ahn. TransDreamer: Reinforcement Learning with Transformer World Models. arXiv:2202.09481 / OpenReview, 2022. (Introduces TSSM, a transformer state-space world model.)